Tellurium Nitro

What changed from regular Tellurium, how the new engine is built, and why it is faster

specJIT | plain-language edition | November 2026

The one-paragraph version. Tellurium is a Python tool biologists use to simulate how chemicals in a cell change over time. All the real number-crunching inside it is done by a C++ engine called *roadrunner*. Tellurium Nitro replaces that engine's heavy lifting with a new engine written in the Rust programming language. The new engine solves the same equations, but it does far less wasted work, it calculates exact answers where the old one used approximations, it uses every processor core at once, and it now translates each model into native machine code (the "JIT"). It also reads model files itself, so loading a big model no longer waits for roadrunner to compile it. Your Python code does not change: the same `te.loada()`, `r.simulate()` and `r.steadyState()` calls just come back faster. If the new engine ever meets something it cannot handle, it quietly hands the job back to the original engine.

Final benchmark (Apple M5 Pro, same Python and roadrunner version for both; full tables in section 9):

Task	Regular tellurium	Nitro	Faster by
Find the steady state (40-chemical model)	244 ms	0.048 ms	5,133x
Control coefficients (how much each reaction controls the flow)	590 ms	0.039 ms	14,979x
Control analysis for 200 parameter sets	251 s	0.014 s	18,165x
Sensitivities over time (86 parameters)	9.3 s	0.022 s	419x
Simulate a time course (40-chemical model)	1.14 ms	0.34 ms	3.4x
Roadrunner's own benchmark models (time courses)	0.02-5.3 ms	0.015-2.8 ms	1.4-2.4x
Load a 1,000-reaction model + first simulate + steady state	over 10 s	0.28 s	about 40x+

The biggest wins are in steady states and control analysis, because that is where the old engine wasted the most work. Plain time-course simulation gains less because the old engine was already good at it; the JIT is what closes the remaining gap there.

1. The three versions

Version	What it is	What does the math
Regular tellurium	The official release from the Sauro lab (sys-bio/tellurium), installed with pip.	roadrunner (C++) for everything.
Tellurium Turbo	Our first fork. Adds te.controlCoefficients(), which runs many parameter sets in parallel worker processes, and optional GPU math (te.gpu).	Still roadrunner (C++) for every individual calculation; only the batching is new.
Tellurium Nitro	This version. Keeps everything from Turbo and adds a new engine written in Rust, plus the JIT compiler.	The Rust engine for almost everything; roadrunner only as a fallback.

2. A quick picture of the problem being solved

A biological model is a list of chemical reactions, for example "S1 turns into S2 at a speed of k1 x S1". Each reaction has a **rate law** (a small formula) and some **parameters** (numbers like k1). From those, the software builds a set of equations saying how fast each chemical's amount is changing. Then it answers questions:

- **Time course:** start from some amounts and step forward in time to see how they change (like a movie).
- **Steady state:** find the point where nothing changes any more (where the movie ends up).
- **Control analysis (MCA):** if I nudge one reaction, how much does each chemical level or flow change? Biologists use this to find which enzymes really control a pathway.
- **Sensitivities:** how much does the whole time course shift if I nudge each parameter?
- **Stochastic simulation:** when there are only a few molecules, simulate random individual reaction events (the Gillespie method).

All of these boil down to evaluating the rate-law formulas, and their slopes (derivatives), millions of times. How quickly and how cleverly you do that decides the speed.

3. Why regular tellurium is slow

We read roadrunner's source code and timed it. The slowness is not because C++ is slow; it comes from how the work is organised:

What roadrunner does	Everyday analogy	Cost
Every steadyState() call can rebuild and re-compile the whole model, sometimes twice.	Rebuilding your kitchen every time you want to cook dinner.	Tens to hundreds of milliseconds per call.
Slopes (elasticities) are found by "finite differences": change each value a tiny bit, re-run, and see what moves.	Finding how steep a hill is by taking a step in every direction and measuring your height each time.	About 170,000 extra model resets on a 40-chemical model; also only approximately correct (errors around 1e-8).
Control-coefficient calls re-solve the steady state every time, even if it was just solved.	Re-measuring a room for every piece of furniture.	The same expensive work repeated.
Its steady-state solver keeps shared global memory, so two copies cannot run at the same time.	One checkout lane in a big supermarket.	Cannot use more than one CPU core at once.

4. What Nitro does instead (the big ideas)

4.1 Exact slopes by algebra, not by trial. When the model is loaded, Nitro works out the derivative of every rate law symbolically, the way you would in a calculus class (the derivative of k1 x S1 with respect to S1 is k1). So the "how steep is the hill" question is answered with a formula, instantly and exactly, instead of 170,000 test steps. This is the single biggest reason steady-state and control analysis are thousands of times faster, and also why Nitro's answers are *more* accurate than the original.

4.2 Build once, reuse forever. The model is converted once and kept in memory. Later calls just plug in new numbers. Changing a parameter does not trigger a rebuild; only changing the model's structure (adding a reaction, for example) does.

4.3 Control analysis in one shot. Control coefficients come from a known matrix formula, $C = -L\,(N_R\,E\,L)^{-1}\,N_R$. Nitro computes the pieces once at the steady state and solves it in a single step, instead of re-solving the model for every coefficient.

4.4 Every core at once. The Rust engine has no shared global memory, so it can run thousands of parameter sets side by side on all CPU cores, without Python's one-thing-at-a-time lock (the GIL) getting in the way.

4.5 Native machine code (the JIT, new in this step). Until now the Rust engine stored each formula as a "tape": a list of tiny instructions ("load k1, load S1, multiply..."), read one by one every time. That is like reading a recipe out loud step by step each time you cook. The **JIT** ("just-in-time compiler") reads the tape once when the model is loaded and translates it into the processor's own native instructions, using an existing Rust compiler library called **Cranelift**. After that, the formulas run at full hardware speed with no reading overhead. Roadrunner does the same thing with a different library (LLVM), so this removes the last place where it still had an advantage. Compiling takes milliseconds and happens once per model; for tiny one-off runs where compiling would cost more than it saves, Nitro simply keeps using the tape.

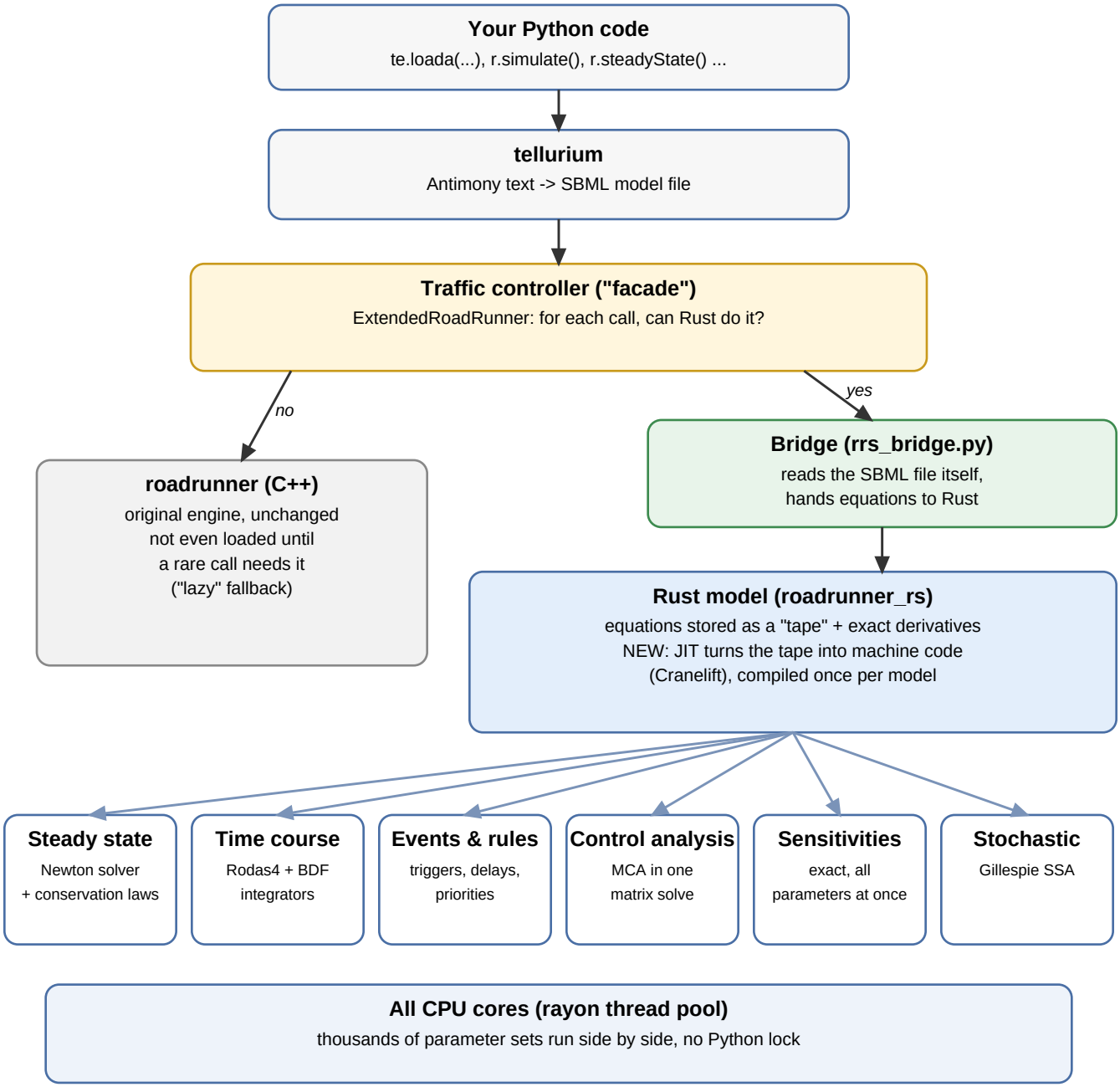
4.6 Smarter time stepping. For time courses Nitro has two integrators (step-forward methods): **Rodas4**, good for normal accuracy, and a variable-order **BDF** method (the same family roadrunner's CVODE uses) for very high accuracy. It picks one automatically from the accuracy you ask for, reuses expensive calculations between steps, and orders its matrix math to avoid unnecessary work ("fill-reducing ordering").

4.7 Loading without waiting for roadrunner. Roadrunner compiles every model into machine code the moment it is loaded. For a 1,000-reaction model that takes 10 seconds or more, even if you only want one quick answer. Nitro now reads the model file itself (with libsbml, the standard SBML reader), works out the starting values, and builds its own engine in a fraction of a second. Roadrunner is still installed, but it is only woken up ("lazy loading") the first time you ask for something Nitro does not do itself, such as editing the model's structure or exporting to another format. When that happens, it is handed the current state, so results carry on seamlessly. Loading plus a first simulation of a 1,000-reaction model went from 13.4 s to 0.28 s.

4.8 It never breaks your code. A "traffic controller" class sits in front of the engine. For each call it asks: can Rust do this exactly the way roadrunner would? If yes, Rust does it; if not, the call goes to the original roadrunner untouched. You can also switch the new engine off: `te.turbo(False)` or the environment variable `TELLURIUM_ENGINE=roadrunner`.

5. How it is built (architecture)

The diagram shows what happens when your code calls, for example, `r.steadyState()`.



Results go back up the same path as normal tellurium / numpy arrays.

Step by step:

- Your code calls tellurium as usual. tellurium turns Antimony text into a standard SBML model file.
- The **bridge** reads that SBML file directly and builds the Rust model; roadrunner is *not* asked to compile it. The **traffic controller** (ExtendedRoadRunner) then answers your calls from the Rust engine, and only wakes roadrunner up for rare calls it does not handle.
- The **Rust model** holds every formula as a tape, its exact derivatives, the conservation laws (quantities that never change, like total enzyme), events and rules. The JIT compiles the tape into machine code.
- The right **engine** runs: steady-state solver, time-course integrator, events, control analysis, sensitivities or stochastic simulation. Batches fan out across all CPU cores.
- Results come back in the same form as before (numpy arrays with column names), so everything else in tellurium (plots, selections, getters) keeps working exactly as before.

6. The building blocks, one by one

Part (file)	What it does, in plain words
Expression tape (rrs/src/expr.rs)	Stores each formula as a compact list of steps; works out exact derivatives by algebra; pre-computes pieces that only depend on fixed parameters.
JIT compiler (rrs/src/jit.rs) - NEW	Turns the tape into native machine code with Cranelift, once per model. Falls back to the tape for anything it does not translate.
Model core (rrs/src/model.rs)	The engines: Newton steady-state solver with conservation laws; Rodas4 and BDF integrators; sparse matrix solver; events (delays, priorities, persistence); rules; Gillespie; control analysis.
Sensitivities (rrs/src/sens.rs)	Tracks how the whole time course shifts for each parameter, using exact derivatives, one parameter per CPU core.
Python bindings (rrs/src/py.rs)	Lets Python call the Rust engine (via PyO3 and maturin), passing numpy arrays without copying where possible.
Bridge (tellurium/analysis/rrs_bridge.py)	Reads the SBML file itself (species, reactions, compartments, rules, events, initial assignments, unit conversions), computes starting values, and builds the Rust model.
Traffic controller (tellurium/roadrunner/extended_roadrunner.py)	Answers calls from Rust; loads the original roadrunner lazily, only for rare calls, and hands it the current state.
Loaders (tellurium/tellurium.py)	te.loada / te.loadSBMLModel create the lazy model instead of compiling it in roadrunner.
Batch control analysis (tellurium/analysis/gpu.py)	te.controlCoefficients(): thousands of parameter sets in one call; uses the Rust engine first, falls back to parallel roadrunner workers, optional GPU.

7. What runs on the new engine

Everything that does real computation and that roadrunner itself supports:

Area	Covered by the Rust engine
Time courses	simulate(), with your accuracy settings; variable-step and fixed-step output; amounts, concentrations, rates of change, initial values, boundary chemicals, rule values.
Model features	Reactions, rate rules, assignment rules, events (with delays, priorities, persistence, "use values from trigger time", and events still waiting between separate simulate() calls), compartments whose size changes, unit conversion factors, stoichiometry that changes over time, function definitions, avogadro and rateOf.
Steady state	steadyState() and all its variants, including models with conservation laws and rate rules.
Control analysis	All scaled and unscaled control-coefficient and elasticity matrices, getCC / getuCC, Jacobians, eigenvalues.
Sensitivities	timeSeriesSensitivities(), including models with events and first-order time derivatives.
Stochastic	Gillespie simulation with seeds, fixed or variable output steps, events.
Batches	te.controlCoefficients() and direct batch calls over thousands of parameter sets.

What still goes to the original roadrunner, and why:

Roadrunner is not loaded at all for normal work. It is loaded once, on demand, for:

- **Editing the model's structure** (addReaction and similar) and exporting to CellML / Matlab.
- **Link and conservation matrices, and conservedMoietyAnalysis = True:** these must pick the same "independent" chemicals as roadrunner's structural analysis, which Nitro cannot reproduce exactly in every case.
- **Integrators other than CVODE and Gillespie** (for example Euler or RK4).
- **getFrequencyResponse:** we could not reproduce roadrunner's exact formula, so there is no trustworthy reference to check a new version against.
- **Features roadrunner itself refuses** (delay equations, algebraic rules, "fast" reactions): roadrunner rejects these models when loading, so they never reach the engine.
- **Random-number functions inside formulas** (the SBML "distrib" package) and second-order sensitivities.

8. How we know the answers are right

- Every Rust feature has an automated test that runs the same model through the original roadrunner and compares the numbers (tellurium/tests/test_turbo.py, rrs/check.py, Rust unit tests).
- Roadrunner's own benchmark models (the "sosbench" set from the official roadrunner repository) are run through both engines and compared.
- The full tellurium test suite gives the same results with the new engine on and off.
- The final benchmark compares all three versions against a high-precision reference (very tight tolerance), so it can say who is closer to the true answer, not just whether they agree.

• Where answers differ, we checked why. Steady states and control coefficients differ by about 5e-8 because roadrunner's trial-step slopes are approximate; tighten roadrunner's step and the gap shrinks to 2e-12. We also found a few cases where roadrunner itself is wrong (for example sensitivities when an event changes a value, or uninitialised numbers in getDependentRatesOfChange); Nitro gives the correct values there.

9. Final benchmark results

Apple M5 Pro (18 cores). Regular tellurium = PyPI release; Turbo = our earlier fork; Nitro = this version. Time is per call; "error" is the distance from a very-high-precision reference answer (smaller is better).

Roadrunner's own benchmark models (sosbench, from the official roadrunner repository):

Model	Regular	Turbo	Nitro	Nitro vs regular	Error regular / Nitro
00009	1.41 ms	1.47 ms	0.70 ms	2.0x	2e-11 / 2e-11
00014	1.50 ms	1.37 ms	0.62 ms	2.4x	5e-6 / 6e-6
00022	5.32 ms	5.31 ms	2.77 ms	1.9x	2e-5 / 3e-5
00033	0.89 ms	0.89 ms	0.40 ms	2.2x	2e-9 / 3e-10
10001	21.9 us	21.6 us	15.2 us	1.4x	9e-6 / 3e-5
10000	fails	fails	109 ms	only Nitro finishes	- / 1e-8
repressilator	fails	fails	111 ms	only Nitro finishes	- / 1e-8

The 40-chemical model from Tellurium Turbo:

Task	Regular	Turbo	Nitro	Nitro vs regular	Nitro vs Turbo
Steady state	244 ms	243 ms	0.048 ms	5,133x	5,100x
Control coefficients	590 ms	581 ms	0.039 ms	14,979x	14,700x
Simulate	1.14 ms	1.14 ms	0.34 ms	3.4x	3.4x
Sensitivities	9.3 s	9.2 s	22 ms	419x	415x
200 parameter sets (MCA)	251 s	8.2 s	14 ms	18,165x	590x

Accuracy: control coefficients are about 1,000x more accurate than roadrunner's (1e-11 vs 4e-8) because Nitro uses exact derivatives; time courses are within roadrunner's own error range. Tellurium's own test suite (385 tests) gives identical results on all three versions; with lazy loading, Nitro passes one more test than roadrunner (a model where roadrunner's solver gives up).

Why some numbers are 2x and others are thousands of times: when both programs do the same work (playing a model forward in time), Nitro is just leaner, about 2-3x. Where roadrunner takes a big detour (rebuilding the model, trial-and-error slopes, one core), Nitro skips almost all of the work, giving 400 to 18,000x.

10. Everything that changed, file by file

File	Status	Purpose
rrs/ (whole folder)	new	The Rust engine (roadrunner_rs): Cargo.toml, src/expr.rs, src/jit.rs, src/model.rs, src/sens.rs, src/py.rs, src/lib.rs, check.py.
tellurium/analysis/rrs_bridge.py	new	Reads SBML directly and builds the Rust engine model.
tellurium/roadrunner/extended_roadrunner.py	changed	Traffic controller, lazy roadrunner; te.turbo() switch.
tellurium/tellurium.py	changed	Model loaders create the lazy (no-compile) model.
tellurium/visualization/sbmldiagram.py	changed	Removed an old Python 2 import that broke import tellurium.
setup.py	changed	Optional "gpu" extra (PyTorch).
tellurium/analysis/gpu.py	new in Turbo, changed	te.controlCoefficients, te.gpu; now tries Rust first.
tellurium/__init__.py	changed	Exports te.turbo, te.gpu, te.getGPU, te.controlCoefficients.
tellurium/tests/test_turbo.py	new	Rust vs roadrunner comparison tests, including sosbench.
tellurium/tests/test_gpu.py	new in Turbo, changed	Batch control analysis tests incl. fallback.
benchmarks/ (bench.py, run.py, tests.py, sosbench/)	new	Speed, efficiency and accuracy benchmark of regular vs Turbo vs Nitro; ONLY=nitro reruns Nitro alone.
docs/specJIT.pdf	new	This document.
docs/roadrunner-rs-spec.md	new	Technical rewrite specification with source references.
README.md	changed	Build steps, benchmark results, limitations.

11. Glossary

Term	Meaning
Rust	A modern, fast programming language that is safe to run on many cores at once.
roadrunner	The original C++ simulation engine inside tellurium.
SBML / Antimony	Standard file format for biological models / a human-friendly way to write them.
Rate law	Formula for how fast one reaction runs.
Steady state	The point where no amounts change any more.
Jacobian	Table of how every rate changes when every amount changes; needed by fast solvers.
Elasticity	How sensitive one reaction's speed is to one chemical's level.
Control coefficient (MCA)	How much a whole pathway's output depends on one reaction.
Finite differences	Estimating a slope by trying small steps; slow and approximate.
Symbolic derivative	The exact slope formula, worked out by algebra.
Integrator	The method that steps a model forward in time (Rodas4, BDF, CVODE).

Term	Meaning
Newton's method	Fast way to solve equations by repeatedly following the slope.
Conservation law	A total that never changes (e.g. free + bound enzyme).
Gillespie / SSA	Simulation of random individual reaction events.
Tape	The engine's compact step-by-step list representing a formula.
JIT (just-in-time compiler)	Turns the tape into the processor's native instructions while the program runs.
Cranelift / LLVM	Code-generator libraries; Nitro uses Cranelift, roadrunner uses LLVM.
Facade / traffic controller	The layer that routes each call to Rust or to the original engine.
GIL	Python's lock that lets only one thread run Python code at a time.
libsbml	The standard library for reading and writing SBML model files.
Lazy loading	Not doing an expensive step (here: compiling the model in roadrunner) until something needs it.